ADVANCED DEEPFAKE DETECTOR

RASHID IQBAL

AI RED TEAMER

DOCUMENTATION

Detect Deception Before It Spreads

ADVANCED DEEPFAKE DETECTOR

In this project, I built an **Advanced Deepfake Detection Application using Python** that analyzes images to determine whether they are real or artificially manipulated. The system performs multiple layers of **digital forensic analysis** such as metadata inspection, error level analysis, frequency domain inspection, noise pattern detection, color distribution analysis, compression artifact detection, and facial consistency checks.

I developed a **graphical user interface (GUI)** using the **Tkinter library**, which allows users to upload an image and run automated forensic analysis. The system processes the uploaded image through several detection techniques and produces a probability score indicating whether the image is **real**, **suspicious**, or **fake**.

To improve security, I also implemented a **secure authentication system** that requires users to log in before accessing the application. Passwords and recovery answers are protected using **cryptographic hashing techniques**, ensuring that sensitive credentials are stored safely.

This project gave me practical experience with **image forensics**, **digital signal processing**, **GUI development**, **authentication systems**, and **cybersecurity concepts**. It also helped me understand how deepfake technology works and how forensic tools can be used to detect digital manipulation.

OBJECTIVE:

The main goals of this project were to:

1) Detect manipulated or deepfake images using multiple forensic analysis techniques.
2) Build an interactive GUI for uploading & analyzing images.
3) Implement secure authentication with password hashing & recovery options.
4) Combine multiple detection algorithms to improve reliability & confidence in results.
5) Learn how images forensics techniques can identify digital manipulation.

KEY CONCEPTS LEARNED:

TKINTER GUI DEVELOPMENT:

- I used the **Tkinter library** to create the **graphical interface** of the application.
- The interface allows users to **upload images**, **run detection**, and **view detailed analysis results**.
- I implemented **buttons**, **menus**, **canvases**, and **text widgets** to create an **interactive environment**.
- This helped me **understand how desktop applications can be built using Python**.

IMAGE FORENSIC ANALYSIS:

- The application analyzes images using several forensic techniques to **detect manipulation**.
- These include **metadata inspection**, **noise analysis**, **frequency analysis**, and **color distribution checks**.
- Each method contributes to a **final probability score that determines whether the image is real or fake**.
- This **taught me how forensic investigators analyze digital media for authenticity**.

ERROR LEVEL ANALYSIS (ELA):

- **Error Level Analysis** identifies area of an image that have different compression levels.
- I recompressed the image at different quality levels & compared pixel differences.

- Significant differences may indicate **image editing or manipulation**.
- This method is **commonly used in digital forensic investigations**.

FREQUENCY DOMAIN ANALYSIS:

- I applied **Fast Fourier Transform (FFT)** to analyze the image in the frequency domain.
- Natural images have predictable frequency distributions.
- Deepfake or edited images often produce unusual frequency patterns.
- This introduced me to **digital signal processing techniques** in image analysis.

NOISE PATTERN ANALYSIS:

- Digital cameras produce natural patterns.
- I analyzed noise levels & inconsistencies across different image blocks.
- If noise patterns vary significantly, it may indicate **image manipulation**.
- This technique helps **identify artificially generated** or **altered images**.

FACE CONSISTENCY ANALYSIS:

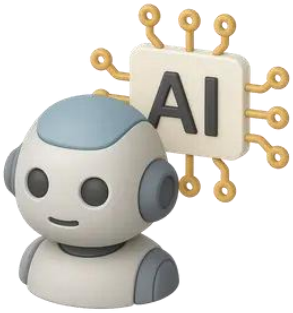
- I used **OpenCV's face detection algorithms** to analyze faces within images.
- The system checks **facial symmetry & smoothness** to **identify unnatural patterns**.
- Deepfake faces often appear overly smooth or asymmetrical due to AI generation artifacts.
- This **helped me understand facial analysis techniques used in AI detection systems**.

METADATA (EXIF) FORENSICS:

- Images contain metadata such as **camera model**, **creation date**, and **editing software**.
- I extracted **EXIF data** to detect whether the image was edited using software like **Photoshop** or **GIMP**.
- Missing or inconsistent metadata may indicate **image tampering**.
- This **taught me how investigator examine digital file properties**.

AUTHENTICATION & SECURITY:

- I implemented a **login system with password protection** before accessing the application.
- Passwords are stored using **PBKDF2 hashing with salt**, which improves security.
- I also added a **security question recovery system** to allow users to **reset credentials safely**.
- This **improved my understanding of secure authentication mechanisms in software development**.



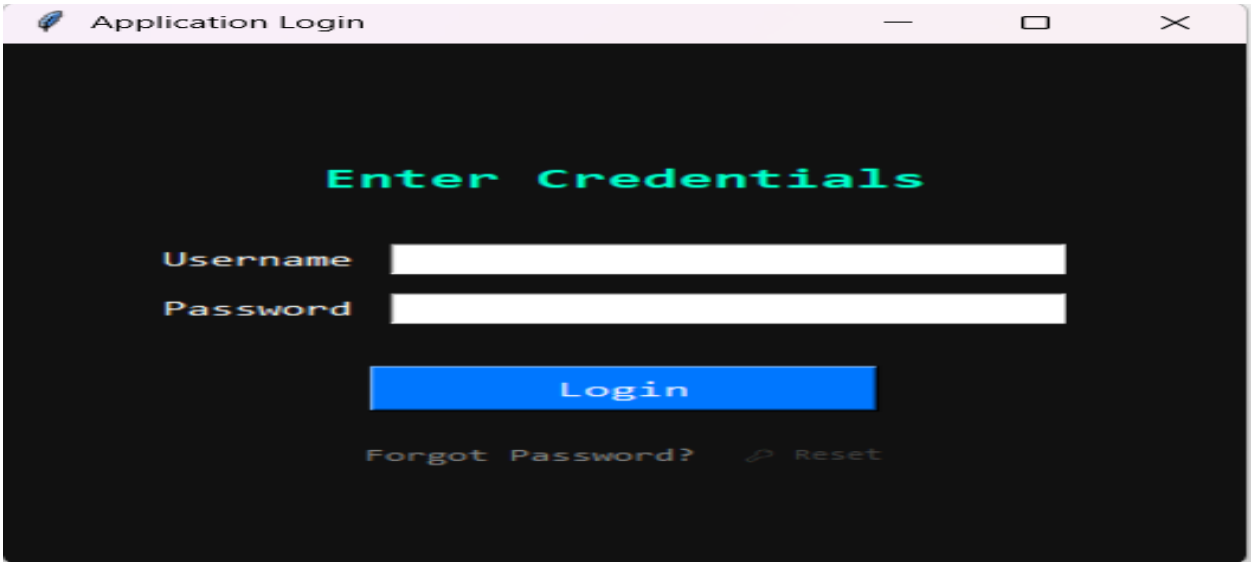
PYTHON IMPLEMENTATION:

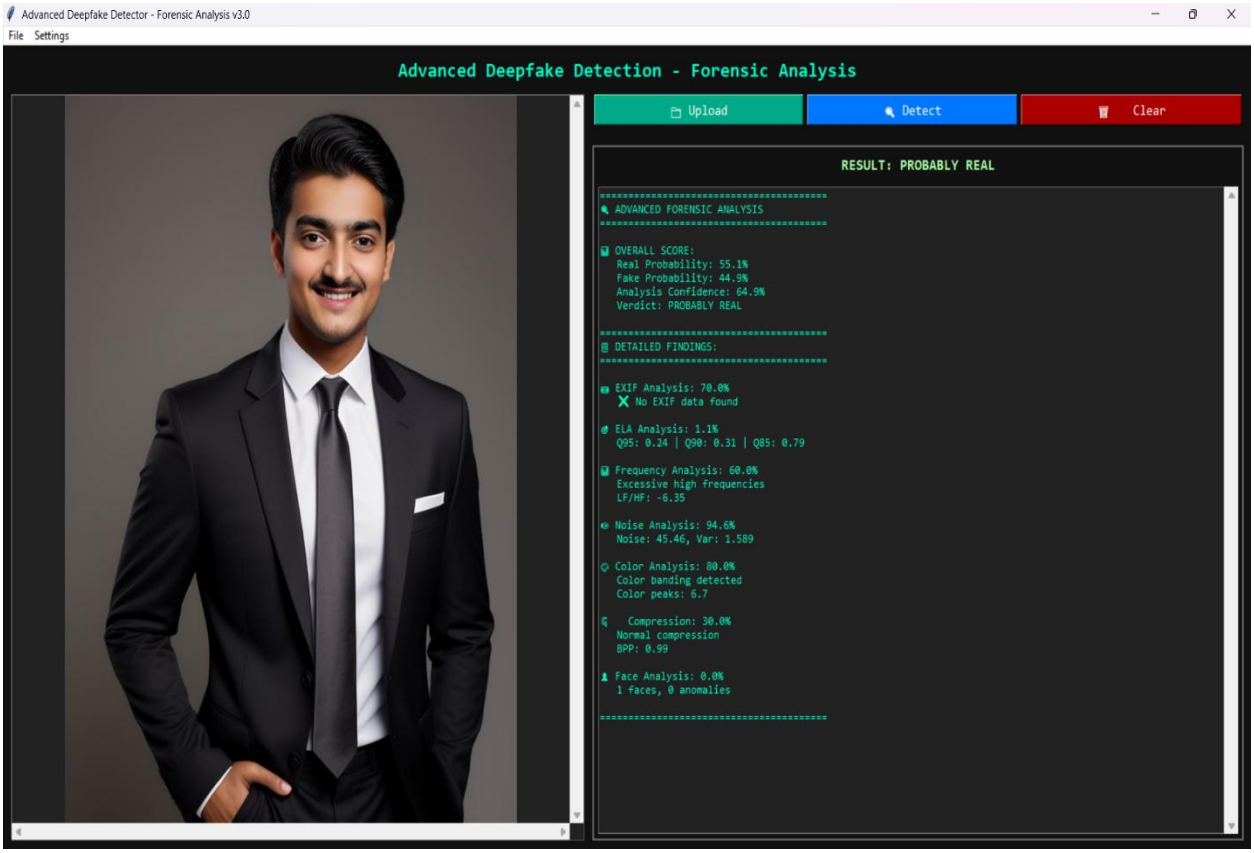
- I used **Tkinter** to build the GUI for uploading & analyzing images.
- I implemented an **authentication system** that:
  - Creates credentials on first run.
  - Stores passwords securely using cryptographic hashing.
  - Allows password recovery through a security question.
- The program allows the user to upload an image using a file dialog.
- Once an image is selected, the system performs several forensic analyses:
  - **EXIF Metadata Analysis** checks camera information & editing history
  - **Error Level Analysis (ELA)** detects compression inconsistencies
  - **Frequency Analysis** analyzes spectral patterns using FFT
  - **Noise Pattern Analysis** examines noise consistency across the image
  - **Color Histogram Analysis** detects unnatural color distributions
  - **Compression Artifacts Analysis** checks for unusual compression ratios
  - **Face Detection & Symmetry Analysis** evaluates facial anomalies
- Each analysis produces a score indicating the probability of manipulation.
- I combined these scores using **weighted calculations** to determine the final fake probability.
- The application then displays:
  - Real or fake probability percentage.
  - Confidence level of analysis.
  - Detailed forensic findings.

This structure allows the program to perform **multi-layered forensic analysis** before producing a final verdict.

PRACTICAL USE:

- This project helped me understand **how manipulated images can be detected using forensic techniques**.
- I gained experience in **image processing & digital forensics**.
- It improved my knowledge of **Python GUI & authentication systems**.
- It shows how analysts can **detect manipulated media**.
- It highlights the need for **tools that detect AI-generated content**.





CONCLUSION:

Exploring the **hidden world of digital image forensics** taught me valuable skills & insights…

I

Built a Advanced Deepfake Detection system to analyze images using forensic techniques.
Gained **hands-on experience** with image processing & computer vision in Python.
Learned to implement secure authentication with password hashing.
Developed a **user-friendly GUI** for **image analysis**.
Strengthened my knowledge in **Python programming**, **Cybersecurity**, and **Digital Forensics**.
Prepared myself for **real-world tasks** in **AI security & forensic investigation**.